

PACKAGES

Scenario 1: CustomerManagement Package

Specification:

CREATE OR REPLACE PACKAGE CustomerManagement AS

```
PROCEDURE AddCustomer(  
    p_CustomerID NUMBER,  
    p_Name VARCHAR2,  
    p_DOB DATE,  
    p_Balance NUMBER  
);
```

```
PROCEDURE UpdateCustomer(  
    p_CustomerID NUMBER,  
    p_Balance NUMBER  
);
```

```
FUNCTION GetCustomerBalance(  
    p_CustomerID NUMBER  
) RETURN NUMBER;
```

```
END CustomerManagement;  
/
```

Body:

CREATE OR REPLACE PACKAGE BODY CustomerManagement AS

```
PROCEDURE AddCustomer(  
    p_CustomerID NUMBER,  
    p_Name VARCHAR2,  
    p_DOB DATE,  
    p_Balance NUMBER  
)  
AS  
BEGIN
```

```
INSERT INTO Customers
VALUES(
    p_CustomerID,
    p_Name,
    p_DOB,
    p_Balance,
    SYSDATE,
    'N'
);
END;
```

```
PROCEDURE UpdateCustomer(
    p_CustomerID NUMBER,
    p_Balance NUMBER
)
AS
BEGIN
    UPDATE Customers
    SET Balance = p_Balance
    WHERE CustomerID = p_CustomerID;
END;
```

```
FUNCTION GetCustomerBalance(
    p_CustomerID NUMBER
)
RETURN NUMBER
AS
    v_Balance NUMBER;
BEGIN
    SELECT Balance
    INTO v_Balance
    FROM Customers
    WHERE CustomerID = p_CustomerID;

    RETURN v_Balance;
END;
```

```
END CustomerManagement;
/
```

```

BEGIN
    CustomerManagement.AddCustomer(
        7,
        'David Miller',
        TO_DATE('1995-02-20','YYYY-MM-DD'),
        12000
    );

    CustomerManagement.UpdateCustomer(7,15000);

    COMMIT;
END;
/
SELECT CustomerManagement.GetCustomerBalance(7)
FROM DUAL;

```

The screenshot displays the FreeSQL web application interface. On the left is a 'Navigator' sidebar with a tree view showing a database schema named 'My Schema' containing several tables: ACCOUNTS, AUDITLOG, CUSTOMERS, EMPLOYEES, LOANS, and TRANSACTIONS. The main area is titled '[SQL Worksheet]*' and contains the PL/SQL code from the previous block. Below the code editor, there are tabs for 'Query result', 'Script output', 'DBMS output', 'Explain Plan', and 'SQL history'. The 'Script output' tab is active, showing the execution results. The output indicates that the 'Package Body CUSTOMERMANAGEMENT' was compiled successfully, with an elapsed time of 00:00:00.014. Below this, the SQL prompt 'SQL>' is followed by the code being executed, and a 'Show more...' link is provided. Finally, the output confirms that the 'PL/SQL procedure successfully completed' with an elapsed time of 00:00:00.016.

```

[ SQL Worksheet ]*
1 BEGIN
2     CustomerManagement.AddCustomer(
3         7,
4         'David Miller',
5         TO_DATE('1995-02-20','YYYY-MM-DD'),
6         12000
7     );
8
9     CustomerManagement.UpdateCustomer(7,15000);

Query result | Script output | DBMS output | Explain Plan | SQL history

Package Body CUSTOMERMANAGEMENT compiled
Elapsed: 00:00:00.014

SQL> BEGIN
      CustomerManagement.AddCustomer(
          7,
          'David Miller',...
Show more...

PL/SQL procedure successfully completed.
Elapsed: 00:00:00.016

```

Query result		Script output	DBMS output	Explain Plan
  Download ▼		Execution time: 0.007 seconds		
		CUSTOMERMANA(		
1		15000		

Scenario 2: EmployeeManagement Package

Specification:

CREATE OR REPLACE PACKAGE EmployeeManagement AS

```

PROCEDURE HireEmployee(
    p_ID NUMBER,
    p_Name VARCHAR2,
    p_Position VARCHAR2,
    p_Salary NUMBER,
    p_Department VARCHAR2
);

```

```

PROCEDURE UpdateEmployee(
    p_ID NUMBER,
    p_Salary NUMBER
);

```

```

FUNCTION CalculateAnnualSalary(
    p_ID NUMBER
) RETURN NUMBER;

```

```

END EmployeeManagement;
/

```

Body:

CREATE OR REPLACE PACKAGE BODY EmployeeManagement AS

```
PROCEDURE HireEmployee(  
    p_ID NUMBER,  
    p_Name VARCHAR2,  
    p_Position VARCHAR2,  
    p_Salary NUMBER,  
    p_Department VARCHAR2  
)  
AS  
BEGIN  
    INSERT INTO Employees  
    VALUES(  
        p_ID,  
        p_Name,  
        p_Position,  
        p_Salary,  
        p_Department,  
        SYSDATE  
    );  
END;
```

```
PROCEDURE UpdateEmployee(  
    p_ID NUMBER,  
    p_Salary NUMBER  
)  
AS  
BEGIN  
    UPDATE Employees  
    SET Salary = p_Salary  
    WHERE EmployeeID = p_ID;  
END;
```

```
FUNCTION CalculateAnnualSalary(  
    p_ID NUMBER  
)  
RETURN NUMBER  
AS  
    v_Salary NUMBER;  
BEGIN
```

```
SELECT Salary  
INTO v_Salary  
FROM Employees  
WHERE EmployeeID = p_ID;
```

```
RETURN v_Salary * 12;  
END;
```

```
END EmployeeManagement;  
/
```

```
BEGIN  
EmployeeManagement.HireEmployee(  
    6,  
    'Kevin',  
    'Analyst',  
    50000,  
    'Finance'  
);  
  
EmployeeManagement.UpdateEmployee(6,55000);
```

```
COMMIT;  
END;  
/
```

```
SELECT EmployeeManagement.CalculateAnnualSalary(6)  
FROM DUAL;
```

FreeSQL | Worksheet | SQL Tutorials

Navigator | Files

My Schema

Tables

Search

ACCOUNTS

AUDITLOG

CUSTOMERS

EMPLOYEES

LOANS

TRANSACTIONS

[SQL Worksheet]*

```

1 SELECT EmployeeManagement.CalculateAnnualSalary(6)
2 FROM DUAL;
3

```

Query result | Script output | DBMS output | Explain Plan | SC

Download | Execution time: 0.007 seconds

	EMPLOYEEMANAG
1	660000

Scenario 3: AccountOperations Package

Specification:

CREATE OR REPLACE PACKAGE AccountOperations AS

```

PROCEDURE OpenAccount(
    p_AccountID NUMBER,
    p_CustomerID NUMBER,
    p_AccountType VARCHAR2,
    p_Balance NUMBER
);

```

```

PROCEDURE CloseAccount(
    p_AccountID NUMBER

```

);

```
FUNCTION GetTotalBalance(  
    p_CustomerID NUMBER  
) RETURN NUMBER;
```

```
END AccountOperations;  
/
```

Body:

CREATE OR REPLACE PACKAGE BODY AccountOperations AS

```
PROCEDURE OpenAccount(  
    p_AccountID NUMBER,  
    p_CustomerID NUMBER,  
    p_AccountType VARCHAR2,  
    p_Balance NUMBER  
)
```

AS

BEGIN

INSERT INTO Accounts

VALUES(

p_AccountID,
 p_CustomerID,
 p_AccountType,
 p_Balance,
 SYSDATE

);

END;

```
PROCEDURE CloseAccount(  
    p_AccountID NUMBER  
)
```

AS

BEGIN

DELETE FROM Accounts

WHERE AccountID = p_AccountID;

END;

```
FUNCTION GetTotalBalance(
```



```

        p_CustomerID NUMBER
    )
    RETURN NUMBER
AS
    v_Total NUMBER;
BEGIN
    SELECT NVL(SUM(Balance),0)
    INTO v_Total
    FROM Accounts
    WHERE CustomerID = p_CustomerID;

    RETURN v_Total;
END;

END AccountOperations;
/

```

The screenshot displays the FreeSQL online SQL editor. The interface includes a top navigation bar with links to 'FreeSQL', 'Worksheet', and 'SQL Tutorials'. On the left, a 'Navigator' pane shows a tree view of a database schema named 'My Schema', with a list of tables: ACCOUNTS, AUDITLOG, CUSTOMERS, EMPLOYEES, LOANS, and TRANSACTIONS. The main editor area, titled '[SQL Worksheet]*', contains a SQL script that defines a package named 'AccountOperations'. The script includes a procedure 'CloseAccount' and a function 'GetTotalBalance'. Below the editor, there are tabs for 'Query result', 'Script output', 'DBMS output', 'Explain Plan', and 'SQL history'. The 'Script output' tab is active, showing the execution of the SQL script. The output indicates that the package 'ACCOUNTOPERATIONS' was successfully compiled, and the elapsed time for the execution was 00:00:00.014 seconds.

```

1  CREATE OR REPLACE PACKAGE AccountOperations AS
10
11      PROCEDURE CloseAccount (
12          p_AccountID NUMBER
13      );
14
15      FUNCTION GetTotalBalance (
16          p_CustomerID NUMBER
17      ) RETURN NUMBER;
18
19  END AccountOperations;
/

```

Query result **Script output** DBMS output Explain Plan SQL history

SQL> CREATE OR REPLACE PACKAGE AccountOperations AS

```

PROCEDURE OpenAccount(
    p_AccountID NUMBER,...

```

[Show more...](#)

Package ACCOUNTOPERATIONS compiled

Elapsed: 00:00:00.014

Query result **Script output** DBMS output Explain Plan SQL history

 

SQL> CREATE OR REPLACE PACKAGE BODY AccountOperations AS

 PROCEDURE OpenAccount(
 p_AccountID NUMBER,...
 Show more...

Package Body ACCOUNTOPERATIONS compiled

Elapsed: 00:00:00.014

```
BEGIN  
    AccountOperations.OpenAccount(  
        108,  
        7,  
        'Savings',  
        25000  
    );  
  
    COMMIT;  
END;  
/
```

FreeSQL | Worksheet | SQL Tutorials

Navigator | Files

My Schema

Tables

Search

ACCOUNTS

AUDITLOG

CUSTOMERS

EMPLOYEES

LOANS

TRANSACTIONS

```

1 BEGIN
2     AccountOperations.OpenAccount (
3         108,
4         7,
5         'Savings',
6         25000
7     );
8
9 COMMIT;
10 END;
11 /

```

Query result | Script output | DBMS output | Explain Plan | SQL history

SQL> BEGIN
AccountOperations.OpenAccount(
108,
7,...
Show more...

PL/SQL procedure successfully completed.

Elapsed: 00:00:00.011

SELECT AccountOperations.GetTotalBalance(7)
FROM DUAL;

Query result		Script output	DBMS output	Explain Plan	SQL history
		Download Execution time: 0.007 seconds			
		ACCOUNTOPERATI			
1		25000			

```

BEGIN
    AccountOperations.CloseAccount(108);

    COMMIT;
END;
/

```

